Handl — how it looks

syntax sheet · every construct as it is written

A module, end to end

```
data Nat :: Zero | Succ(pred : Nat)

data Opt(a : Type) :: None | Some(x : a)

to_num : Nat ~> Number = n ~> match n { Zero: 0, Succ(p): 1 + to_num(p) }

get_or : Opt(Number) ~> Number ~> Number =
  o ~> d ~> match o { None: d, Some(x): x }

export double : Number ~> Number = n ~> n * 2
export point := { x: 1, y: 2 }

result := [to_num(Succ(Succ(Zero))), get_or(Some(41))(0), point.x]
```

Declarations

```
data Nat :: Zero | Succ(pred : Nat)            -- family: ctors after ::

data Vec(n : Nat, a : Type) :: VNil : Vec(0, a)
                             | VCons(x : a, xs : Vec(n, a)) : Vec(n + 1, a)

export double : Number ~> Number = n ~> n * 2  -- exported arrow

helper : Number ~> Number = n ~> n + 1         -- module-local arrow
k := 41                                        -- value binding (inferred)

Nat  := { Zero : Nat, Succ : Nat ~> Nat }      -- ADR-0158: one decl, no `::`

Vec<n, a> := {
  VNil  : Vec<0, a>,
  VCons : (k : Nat, x : a, xs : Vec<k, a>) ~> Vec<Succ k, a>
}

State   := { get : Unit ~> Nat, put : Nat ~> Unit }   -- an effect is the same record
counter : Unit ~> IO<State>                           -- or ~> State if the marker drops

instance Eq(Nat) := { eq: (x, y) ~> x == y }   -- dictionary as a value
```

> `data` today, generator-record next
>
> `data X :: C | D` is the **only spelling the parser accepts today** (`node_kind: data` in `ast`/`grammar`/`eval`/`types`; zero record-form declarations in-tree). ADR-0158 (*Proposed*) deletes it in favour of the generator-record above: a field whose codomain is the former itself is a constructor, a field whose codomain is anything else is an effect op — one mechanism, two folds (`match` / `X.rec` vs `handle`). `IO` then becomes an internal marker. Migration is mechanical and not done.

```
op "hash_set.new" : _ -> Array                -- native op declaration
op "hash_set.add" : (Array * Value) -> Array  -- tuple arguments use *
```

Types

```
a ~> b                          arrow (curried): a ~> b ~> c
```

```
Vec<n, a>                       index application with <...>
f<n, a>                         implicit index binders
lookup<a, ev : Eq(a)>           evidence binder: a dictionary argument

Sigma(Nat, m ~> Vec(m, Bool))   dependent pair, long form
exists m : Nat, Vec(m, Bool)    the same type, sugar form
forall n : Nat, Nat             Pi, sugar form

IO<{ puts }>                    effect row: an op, no result yet
IO<{ puts: Unit, fs }>          handled puts (returns Unit), fs still performed

A ~[2]~> B                      dimension arrow: a 2-cell
Type(0)                         universe
type.unknown                    dynamic top (host results land here)
```

Expressions

```
x ~> x + 1                      an arrow literal
double(21)                      application
add(1)(1)                       curried application

{ x: 1, y: 2 }                  record literal
point.x                         field access
[1, 2, 3]                       array literal
[Zero, VNil]                    dependent pair (Sigma introduction)

if sum == 7 { "seven" } else { "other" }

do {
  v := fst(p)
  { out: v }                    last expression is the block's value
}

match n { Zero: 0, Succ(p): 1 + to_num(p) }
Vec.rec(null, 0, step)(2, Bool, v)     explicit recursor
handle printLn with { puts: h }        discharge an effect
```

Patterns

```
match n { Zero: 0, Succ(p): ... }          constructor with payload
match x { 3: true, _: false }              literal and wildcard
match xs { []: a, [h, ...t]: b }           list, rest with ...
match o { None: d, Some(x): x }            nullary vs payload ctor
match s { [m, ys]: ... }                   dependent pair: m binds the index
match kvs { [[k, v], ...t]: ... }          nested pair pattern
```

One-line legend

`~>` arrow · `::` constructor list · `<...>` indices (erased) or evidence (passed) · `[...]` list or dependent pair · `*` tuple type · `:=` value binding · `_` wildcard.

Keywords are plain words: `data`, `export`, `op`, `instance`, `handle`, `match`, `do`, `while`, `if`, `else`, `forall`, `exists`.